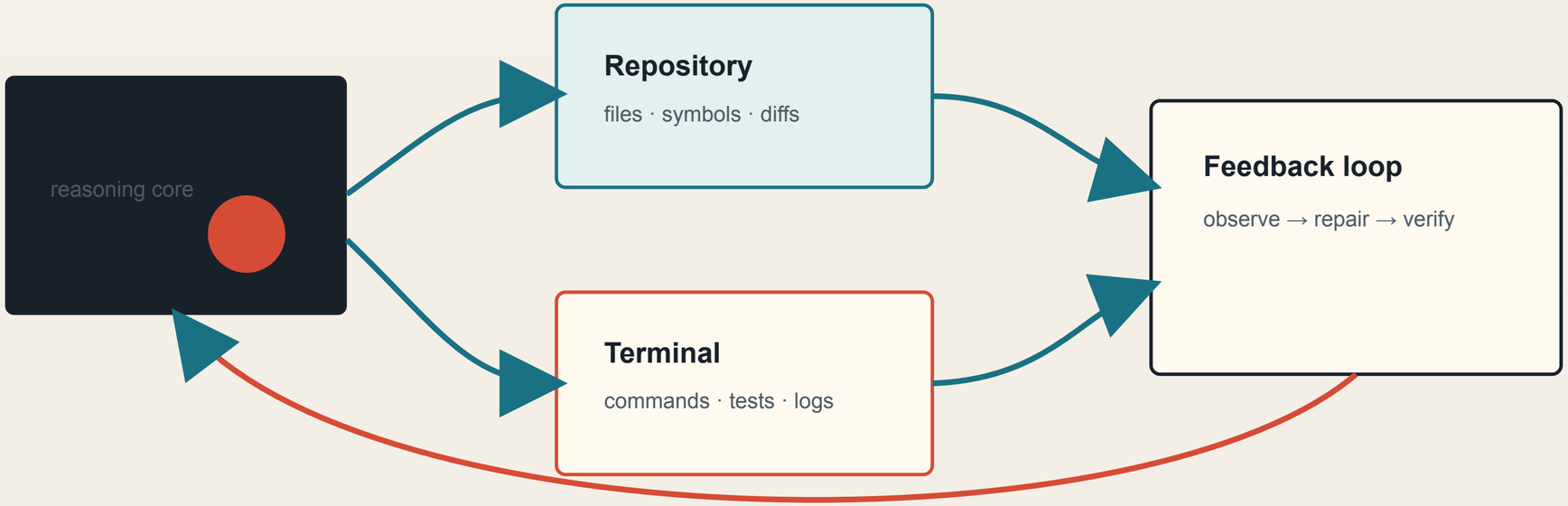
ENGINEERING SYSTEMS

Agentic Harness для написания кода

Как LLM превращается в управляемого coding agent:
контекст, инструменты, sandbox, проверки и обратная связь.



ПРОБЛЕМА

LLM сама по себе не является надежным исполнителем изменений

Chat completion генерирует текст. Agentic coding system управляет состоянием, действиями и проверкой результата.

Chat completion

- Одноразовый ответ
- Нет доступа к repo state
- Не применяет diff
- Не доказывает работоспособность

Agentic coding system

- Итеративный цикл
- Контекст кодовой базы
- Явные инструменты
- Валидация и repair

ОПРЕДЕЛЕНИЕ

Harness — управляющая обвязка вокруг reasoning core

Модель выбирает гипотезы и следующие шаги. Harness превращает их в контролируемые операции с файлами, командами и состоянием.

Reasoning core

LLM анализирует задачу, строит план, объясняет ошибки и предлагает patch.

Execution / control layer

Orchestrator, tools, sandbox, permissions, memory, telemetry и policy gates.

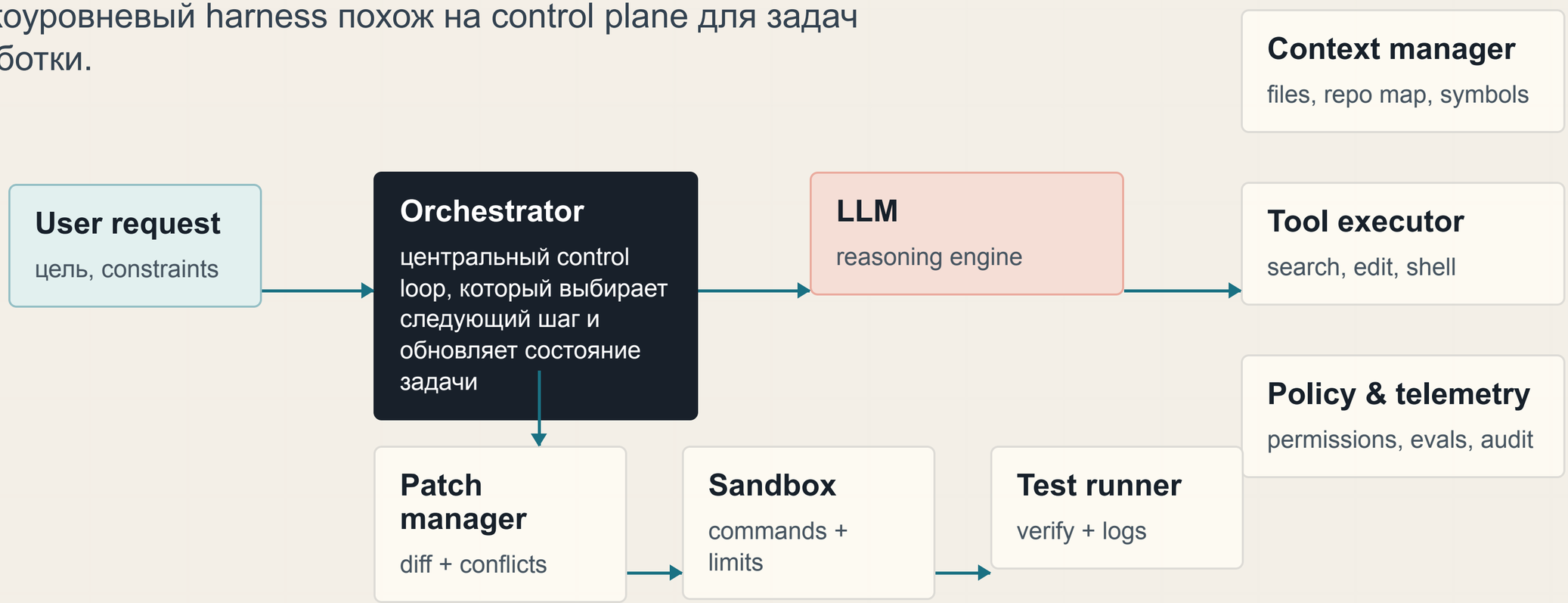
Итог

Агент становится не чат-ботом, а исполняемой системой с наблюдаемым циклом.

АРХИТЕКТУРА

Надежность появляется из связки компонентов, а не из одной модели

Высокоуровневый harness похож на control plane для задач разработки.



CONTROL LOOP

Главный цикл: понять, действовать, наблюдать, чинить

Один ответ модели редко достаточен. Устойчивость дает повторяемый цикл с проверкой после каждого существенного шага.

1. Understand task
решение и действие

2. Build context
решение и действие

3. Plan
решение и действие

4. Act
решение и действие

5. Observe
обратная связь

6. Repair
обратная связь

7. Verify
обратная связь

8. Summarize / PR
обратная связь

```
while task.open:
  context = build_context(repo, task)
  plan = llm.plan(context)
  result = tools.apply(plan.next_action)
  observation = validate(result)
  task.update(observation)
```

CONTEXT MANAGER

Качество контекста часто важнее размера модели

Context manager — слой, который выбирает, сжимает и обновляет информацию, нужную модели прямо сейчас.

01
Чтение файлов

02
Repo map

03
Semantic search

04
Dependency graph

05
Symbol index

06
Recent diffs

07
Issue / PR context

08
User constraints

TOOL EXECUTOR

Инструменты должны быть явными, логируемыми и ограниченными

Tool executor — адаптер между намерением модели и реальным действием в окружении разработки.

`read file` `search` `edit file` `run shell` `run tests` `inspect logs` `package manager` `linter / typechecker` `browser / API / docs lookup`

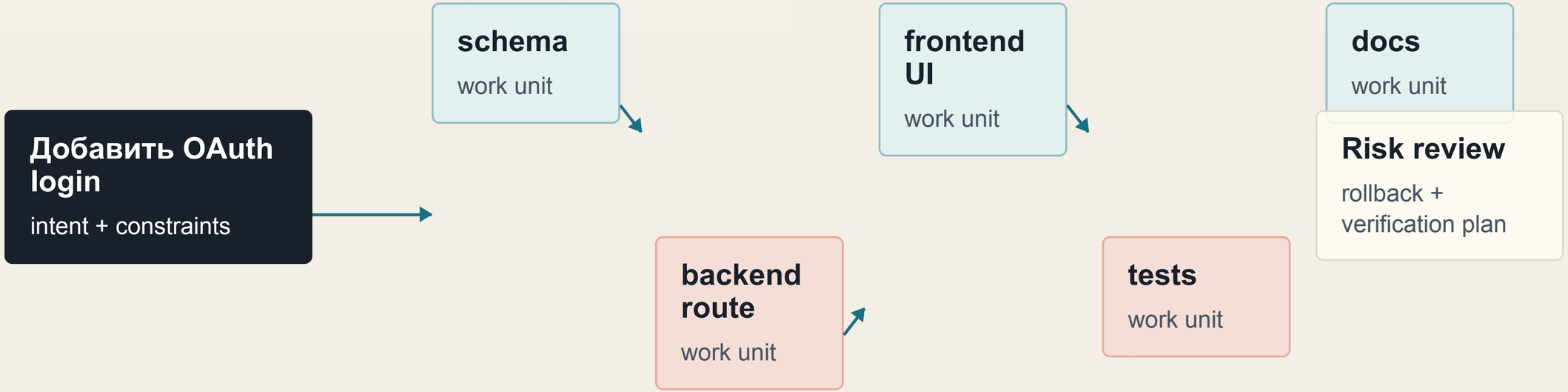
адаптер, который выполняет разрешенные действия и возвращает наблюдения

Каждый вызов инструмента должен иметь вход, результат, статус, длительность и policy context.

PLANNING

Decomposition превращает запрос в проверяемые рабочие единицы

Harness помогает выделить intent, ограничения, затронутые модули, риски и стратегию отката.



PATCH MANAGER

Код меняется маленькими проверяемыми diff-ами

Patch manager отвечает за применение изменений, конфликтность, перечитывание измененных участков и минимальный scope.

Выбрать файлы
change

Сгенерировать patch
change

Применить diff
change

Проверить конфликт
safety check

Перечитать участки
safety check

Сузить scope
safety check

слой применения, проверки и учета diff-изменений

Большой refactor без промежуточной проверки хуже серии маленьких diff-ов с понятным наблюдением после каждого шага.

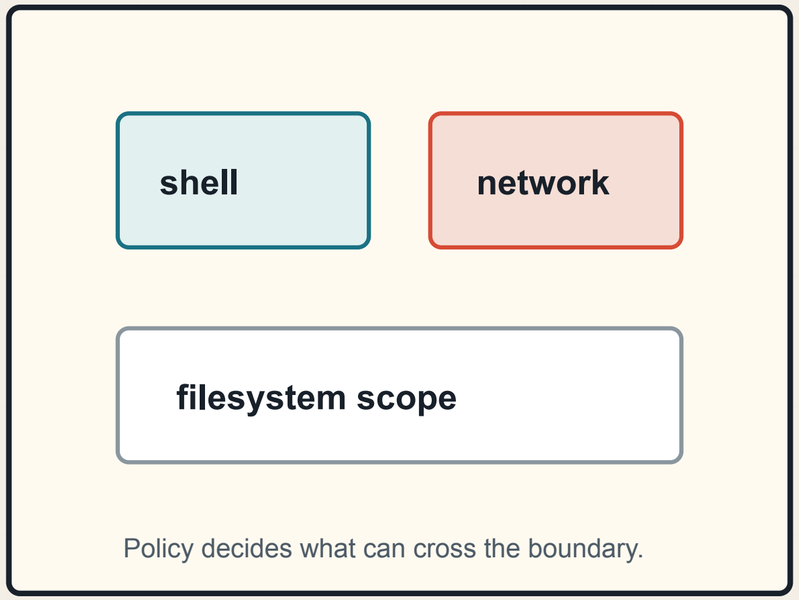
SANDBOX

Coding agent должен быть мощным, но не безграничным

Sandbox — изолированная среда выполнения команд с политиками сети, файлов, секретов и опасных операций.

изолированная среда с лимитами и политиками выполнения

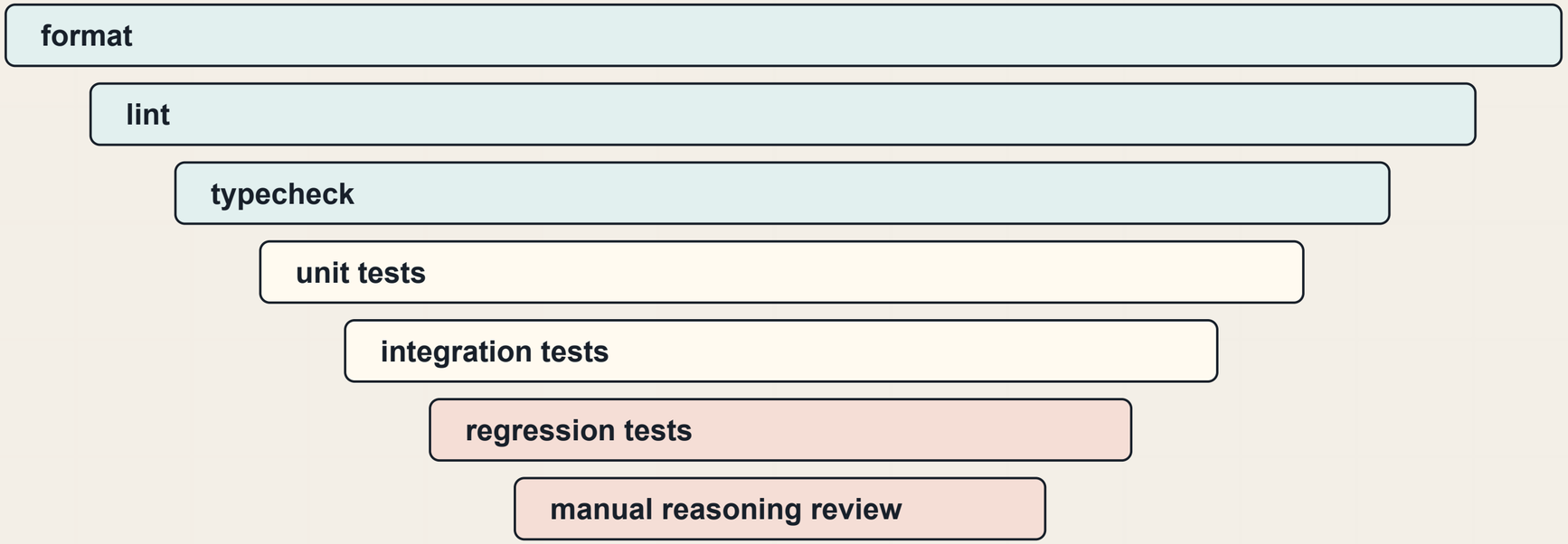
- Изоляция команд
- Лимиты времени и ресурсов
- Allowlist / denylist
- Секреты и credentials
- Network policy
- Approval gates



VALIDATION

Harness должен доказать, что изменение работает

Валидация — это воронка от дешевых локальных проверок к более дорогим интеграционным сценариям.

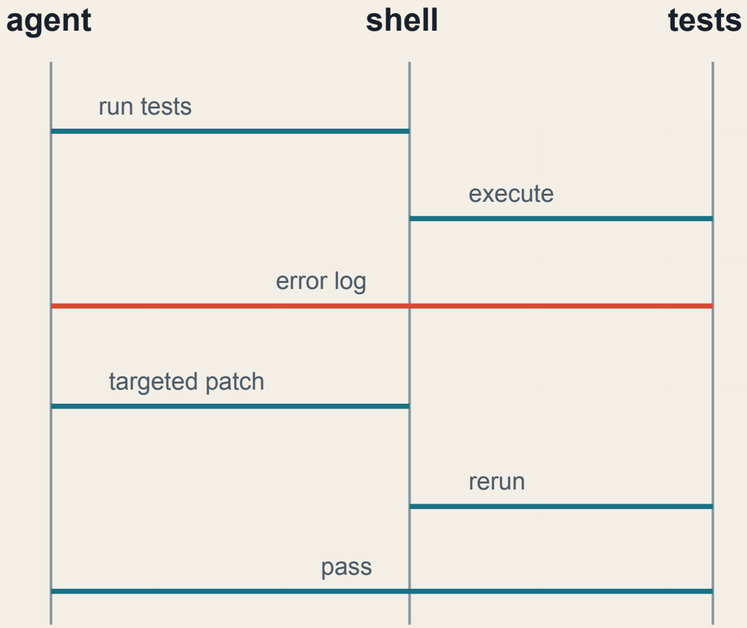


Дальше по воронке: дороже, медленнее, выше уверенность.

SELF-REPAIR

Ошибки становятся входом для следующей гипотезы

Repair loop связывает test failure, анализ логов, targeted patch и повторный запуск проверки.




```
failure = run_tests()
log = summarize(failure.output)
hypothesis = llm.explain(log, changed_files)
patch = llm.targeted_fix(hypothesis)
apply(patch)
rerun_focused_tests()
```

SECURITY & GOVERNANCE

Harness — это also safety boundary

Безопасность строится на разрешениях, журнале действий и явном контроле операций с высоким риском.

gate
Permission gates

02
Human approval

03
Secrets and credentials

04
Destructive commands

05
Supply-chain risk

06
Audit log

07
Policy enforcement

TELEMETRY & EVALS

Без evals агентская система не улучшается управляемо

Evals — повторяемые тесты качества поведения агента; telemetry показывает, где цикл дорогой, хрупкий или требует человека.

TSR
Task success rate

Tests
Tests passed

Iters
Number of iterations

Diff
Patch size

Human
Intervention rate

Regress
Regression rate

PR time
Time-to-PR

Cost
Cost per task

ИТОГ

Пять принципов хорошего coding harness

Хорошая обвязка делает работу агента наблюдаемой, ограниченной и проверяемой.

1. explicit tools

Явные интерфейсы действий

2. tight feedback loop

Быстрая обратная связь

3. scoped context

Контекст под задачу

4. safe execution

Ограниченная мощность

5. measurable quality

Качество через метрики

LLM генерирует гипотезы; harness превращает их в проверяемые изменения кода.